

REDIS

What is Redis?

Redis is a **super-fast database** that stores everything in your computer's memory (**RAM**) instead of on a hard drive.

Think of it like a giant dictionary where you look up values by their keys. Because RAM is much faster than disk, Redis can handle millions of operations per second.

How is Data Stored?

You store things as key-value pairs. **e.g.**, user:123:name "Ismail."

But Redis isn't limited to simple text. You can store

1. Lists (like a shopping list),
2. Sets (unique items with no duplicates),
3. Sorted Sets (ranked items like a leaderboard).
4. Hashes (like a mini-dictionary inside a key).

Why People Use It?

Caching is the big one. Your main database is slow, so you store frequently accessed data in Redis.

User hits your app, you check Redis first, return instantly if found, otherwise query the slow database and save the result to Redis for next time.

- **Reading from an SSD = 100ms - 300ms**
- **Reading from a RAM = 10ns - 30ns**

Session storage is another. When users log in, their session data lives in Redis, so any server can access it.

Rate limiting, real-time leaderboards, job queues, chat systems, all common uses.

Scaling Up Redis:

One Redis server not enough? You can create replicas that copy data from the main server. Replicas handle read requests; the main server handles writes.

Need even more? Redis Cluster splits your data across multiple servers automatically. Each server holds a portion of your keys.

Things to Watch Out For

1. Your data can't be bigger than your RAM. If memory fills up, Redis either rejects new writes or starts deleting old data, depending on your settings.
2. Keys don't disappear on their own. You need to set an expiration time, or they stay forever.
3. Redis is single-threaded. One slow command blocks everything else waiting behind it.

When to Use Redis?

Use it when you need speed and can tolerate occasional data loss. Perfect for caching, sessions, and real-time features.

Don't use it as your only database for important data you can't afford to lose; pair it with something more durable, like PostgreSQL or MongoDB.

Redis Commands

Note: Redis Commands are case-insensitive.

1. **PING:** checks if Redis is running.
2. **INFO:** Detailed Redis server info
3. **SET:** Store a value **SET <key> <value>**
4. **GET:** Retrieve a key **GET <key>**
5. **DEL:** Delete a key **DEL <key>**
6. **EX:** Set time to live (TTL)
SET <key> <value> EX 60 # expires in 60 seconds
SETEX <key> 60 <value>
7. **INCR:** increment the integer by 1 **INCR counter**

LIST (ARRAYS):

1. **LPUSH:** push to left **LPUSH <list_name> item1**
2. **RPUSH:** push to right **RPUSH <list_name> item1**
3. **LPOP:** pop from left and return **LPOP <list_name>**
4. **RPOP:** pop from right and return **RPOP <list_name>**
5. **LRANGE:** give items in range
LRANGE: <list_name> 0 -1 # get all items
6. **LLEN:** Length of the list **LLEN <list_name>**

Sets:

Set Unordered collection of unique values. No duplicates allowed.

```
SADD fruits "apple" "banana" "apple"
SMEMBERS fruits    # returns: apple, banana (no duplicate)
```

Use case: tags, unique visitors, tracking "has this been seen before?"

Commands:

1. **SADD:** add item(s) to a set **SADD <set_name> item1 item2**
2. **SMEMBERS:** get all items in a set **SMEMBERS <set_name>**
3. **SISMEMBER:** check if an item exists in the set **SISMEMBER <set_name> item1**
(returns 1 if exists, 0 if not)

Sorted Sets (ranked data)

Sorted Sets store unique items *with a score*, sorted by that score

1. **ZADD:** add a member with a score
ZADD <zset_name> score1 member1 score2 member2
2. **ZRANGE:** get items by rank (low → high)
ZRANGE <zset_name> start end
e.g. ZRANGE leaderboard 0 -1 — get all
3. **ZREVRANGE:** get items by rank (high → low)

ZREVRANGE <zset_name> start end

e.g. ZREVRANGE leaderboard 0 9 — top 10

Hash:

Key-value pairs under a single Redis key. Like an object/dictionary.

```
HSET user:1 name "John" age 30 city "NYC"
HGET user:1 name    # "John"
HGETALL user:1      # name=John, age=30, city=NYC
```

Use case: storing objects like user profiles, session data, and configs.

Commands:

1. **HSET**: set a field inside a hash **HSET <hash_name> <field> <value>**
2. **HGET**: get the value of a specific field **HGET <hash_name> <field>**
3. **HGETALL**: get the entire hash (all fields + values) **HGETALL <hash_name>**

Utilities:

1. **EXISTS**: Checks if the key exists. **EXISTS <key>**
2. **EXPIRE**: Set time to live (TTL) in seconds. **EXPIRE key 300**
3. **TTL**: Check remaining time to live (TTL). **TTL <key>**
4. **KEYS pattern*** Find all keys with a specific pattern. **KEYS api:products***
5. **SCAN 0 MATCH pattern*** starts a non-blocking search for keys matching pattern*. Super practical in production because it won't nuke performance.

REDIS WITH NODE.JS

```
npm install redis
```

```
import { createClient } from "redis";

const redis = createClient({
  url: process.env.REDIS_URL || "redis://localhost:6379",
  socket: {
    reconnectStrategy: (retries) => Math.min(retries * 50, 500)
  }
});

redis.on("error", (err) => console.error("Redis error:", err));
redis.on("connect", () => console.log("Redis connected"));

export default redis;
```

```
import redis from "../lib/redis.js";
await redis.connect();
```

Cache Invalidation:

Cache invalidation is deciding **when and how to delete or update cached data** so it doesn't become outdated.

The hard part: your cache says X, but the source of truth now says Y. How do you know? When do you act?

Cache Invalidation Strategies:

TTL (Time-to-Live): Data expires after N seconds, simple but dumb.

Used when data validity is predictable. E.g.,

- User Session Information (a 2, 3, or 7-day session)
- Weather data (update hourly)

Disadvantage: The User may see outdated data

Write-through / Write-behind or Direct / Immediate: Update cache when we update the DB.

Best strategy for REST API's

- Minimum chance of serving outdated data.
- Depends on user action (if a product updates, we can programmatically remove all the cached data for `/api/products/get-all-products`)
- Full control over caching.

Disadvantages: Need complex retry logic.

Event-driven: Publish invalidation events when data changes

Manual purge: We explicitly delete keys (error-prone).

The famous quote exists for a reason:

"There are only two hard things in Computer Science: cache invalidation and naming things."

It's hard because distributed systems have no single source of truth at any instant. We are constantly trading off **freshness vs. performance**.

Eviction Policies:

Eviction policies are the rules for deciding what to delete when the cache exceeds available memory.

Cache full → pick a victim → remove it → make room for new data.

Eviction Policies Command:

LRU (Least Recently Used): kick out what hasn't been accessed the longest. Most popular, usually good enough.

LFU (Least Frequently Used): Kick out what's accessed least often. Better for skewed access patterns.

FIFO (First In, First Out): Oldest entry goes first. Simple, often dumb.

TTL-based: Evict expired entries first.

Random: Exactly what it sounds like. Surprisingly, not terrible in some workloads.

Redis uses approximated **LRU/LFU by default**.

TL; DR:

Invalidation = "This data is wrong, remove it."

Eviction = "We're out of space, sacrifice something."

Eviction Policies Commands:

- **CONFIG SET maxmemory 100mb:**
- **CONFIG SET maxmemory-policy noeviction**
 - Non-eviction is by default in non-eviction; we got an error when the memory is full.
 - For **LRU** policy: **allkeys-lru**
 - For **LFU**: **allkeys-lfu**